FIAP ON

BUILDING STATISTICAL MODELS

ANÁLISE MULTIVARIADA DE DADOS - ANÁLISE DE CLUSTER COM PYTHON

5

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Tendência análise de *cluster*.....7
Figura 2 – Exemplo de análise de *cluster* 8
Figura 3 – Conceito distância euclidiana.....11
Figura 4 – Algoritmos de *cluster* hierárquico 13
Figura 5 – Algoritmos de *cluster* não hierárquico 14
Figura 6 – Exemplo dendograma. 15
Figura 7 – Exemplo coeficiente de aglomeração.....16
Figura 8 – Exemplo gráfico WCSS.....17
Figura 9 – Exemplo gráfico da silhueta 17
Figura 10 – Dados análise RFM.....21
Figura 11 – Dados padronizados *Min-Max*.....23
Figura 12 – Dados padronizados *scale* 23
Figura 13 – Dendograma dos dados analisados.....24
Figura 14 – Gráfico coeficientes de aglomeração 25
Figura 15 – Valor dos centroides *cluster* hierárquico 26
Figura 16 – *Clusters* associados aos dados originais 27
Figura 17 – Número de clientes nos *clusters* 28
Figura 18 – Grupos classificados 29

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Ferramentas *Python* análise de *clusters* 19

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Importar módulos iniciais e dados.....20
Código-fonte 2 – Dados dos exemplos 20
Código-fonte 3 – Exemplos padronização das variáveis.....22
Código-fonte 4 – *Numpy* para *dataframe* dados padronizados 22
Código-fonte 5 – Número de *clusters* – Dendograma 24
Código-fonte 6 – Obtenção dos coeficientes de aglomeração 25
Código-fonte 7 – Obtenção dos centroides 26
Código-fonte 8 – Aplicação do *K-Means* 27
Código-fonte 9 – Obtenção das quantidades de grupos 28
Código-fonte 10 – Aplicação de análise qualitativa dos grupos 29

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 ANÁLISE MULTIVARIADA DE DADOS - ANÁLISE DE CLUSTER COM PYTHON 6
1.1 Introdução 6
1.2 Principais conceitos sobre análise de *cluster* 7
1.2.1 Diferenças entre similaridade e correlação 9
1.3 Etapas para realizar uma análise de *cluster* 10
1.3.1 Escolha das variáveis, identificação de *outliers* e padronização. 10
1.3.2 Medida de distância (similaridade). 11
1.3.3 Escolha do algoritmo de agrupamento (hierárquicos e não hierárquicos). 12
1.3.4 Escolha da quantidade de grupos 15
1.3.5 Interpretação dos grupos.....17
2 APLICAÇÕES PRÁTICAS EM ANÁLISE DE CLUSTER.....19
2.1 Principais funções no *Python* 19
2.2 Dados e exemplos.....20
2.3 Padronizar os dados 21
2.4 Obter número de *clusters*: algoritmos hierárquicos 23
2.5 Obter os *clusters*: algoritmos não hierárquicos 26
2.6 Interpretação dos *clusters* 27
ATIVIDADE TREINO – ANÁLISE DE *CLUSTER* 31
REFERÊNCIAS.....33

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 ANÁLISE MULTIVARIADA DE DADOS - ANÁLISE DE CLUSTER COM PYTHON

1.1 Introdução

Em diversas situações nas empresas, as áreas de negócios necessitam categorizar tipos de produtos, definir perfis de usuários ou descobrir nichos de clientes em variados mercados. Diante desses desafios, a área de *Analytics* pode prover análises específicas para auxiliar a área de negócio dessas atividades.

Uma técnica bastante recomendada para ajudar nos objetivos de encontrar perfis de indivíduos em grandes bancos de dados é a análise de *cluster*, também denominada de análise de conglomerados. Esse modelo de *Machine Learning* (ML) não supervisionado tem o objetivo de encontrar grupos em quantidades massivas de dados observando múltiplas características.

Neste capítulo, conheceremos de forma mais clara como o algoritmo pode ser aplicado, os conceitos básicos relacionados à técnica e as etapas a serem seguidas. Além disso, por meio do *Python*, poderemos aplicar um exemplo em dados selecionados.

Na área acadêmica, essa técnica é utilizada há muitos anos. Segundo Hennig *et al.* (2015), o termo “Análise de Cluster” (*Cluster Analysis*) foi utilizado pela primeira vez no trabalho de Robert Tryon em 1939; nessa época, o autor estudava psicologia comportamental.

Depois dessa proposição, áreas de taxionomia, biologia e tipologia começaram a utilizar de forma constante. Perguntas como “O que seria mais parecido com um pato, um corvo ou pinguim?” puderam ser respondidas facilmente ao categorizar as diversas características num banco de dados e aplicar a análise.

Em 1961, Gavin Ross, da Estação Experimental de Rothamsted, no Reino Unido, um dos primeiros centros de pesquisa em agricultura do mundo, iniciou o uso da técnica de forma prática na área de agricultura, principalmente para classificar vermes nematoides, cepas bacterianas e perfis de solo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Depois disso, a técnica tornou-se bastante popular nas diversas áreas de pesquisa e negócios. Na figura “Tendência análise de cluster”, temos um gráfico apresentado em Hennig *et al.* (2015), em que podemos observar um crescimento exponencial entre os anos de 1960 a 2010.

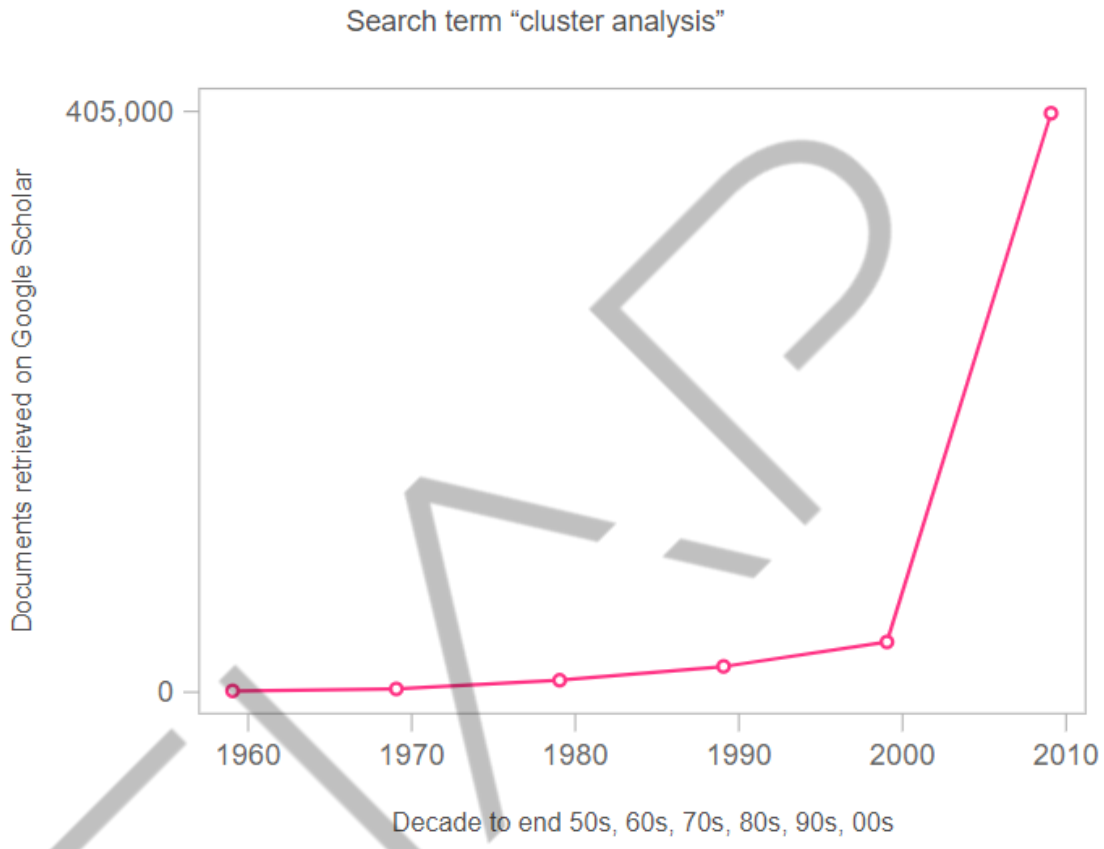

Figura 1 – Tendência análise de *cluster*
Fonte: Hennig *et al.* (2015)

Nos próximos tópicos, explicaremos o conceito da análise de conglomerados, a forma de aplicação em análise de dados e como executar a técnica no ambiente do *Python*.

1.2 Principais conceitos sobre análise de *cluster*

Como produto deste capítulo, no mínimo duas perguntas deverão ser respondidas ao fim da leitura, são elas:

- Qual o conceito e os objetivos da análise de agrupamentos?
- Quais são as etapas de uma análise de agrupamentos?

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Baseando-se em Maroco (2010) e Hair *et al.* (2009), conseguimos definir uma análise de *cluster* como uma técnica de ML não supervisionado que permite agrupar casos em grupos homogêneos em função do grau de similaridade entre os indivíduos por meio das características selecionadas.

Por exemplo, imagine que um analista esteja em atuação numa empresa de *Healthtech* (empresa de tecnologia voltada a resolver problemas na área de saúde) e precise criar perfis de usuários baseando-se nas colunas: “Número de vezes que utilizou o plano de saúde (Qtd)” e “Gastos com plano de saúde (Custo)”.

O analista pode aplicar uma *cluster analysis* e descobrir 3 perfis de usuários:

- Cluster 1 – Alto Custo e Baixo Qtd.
- Cluster 2 – Médio Custo e Médio Qtd.
- Cluster 3 – Alto Custo e Alto Qtd.

Na figura “Exemplo análise de *cluster*”, podemos visualizar como a análise pode categorizar os indivíduos por meio da similaridade entre Custo e Qtd. Dessa forma, as áreas de negócios poderão desenvolver inúmeras estratégias para gerar um melhor relacionamento com os beneficiários dos planos de saúde.

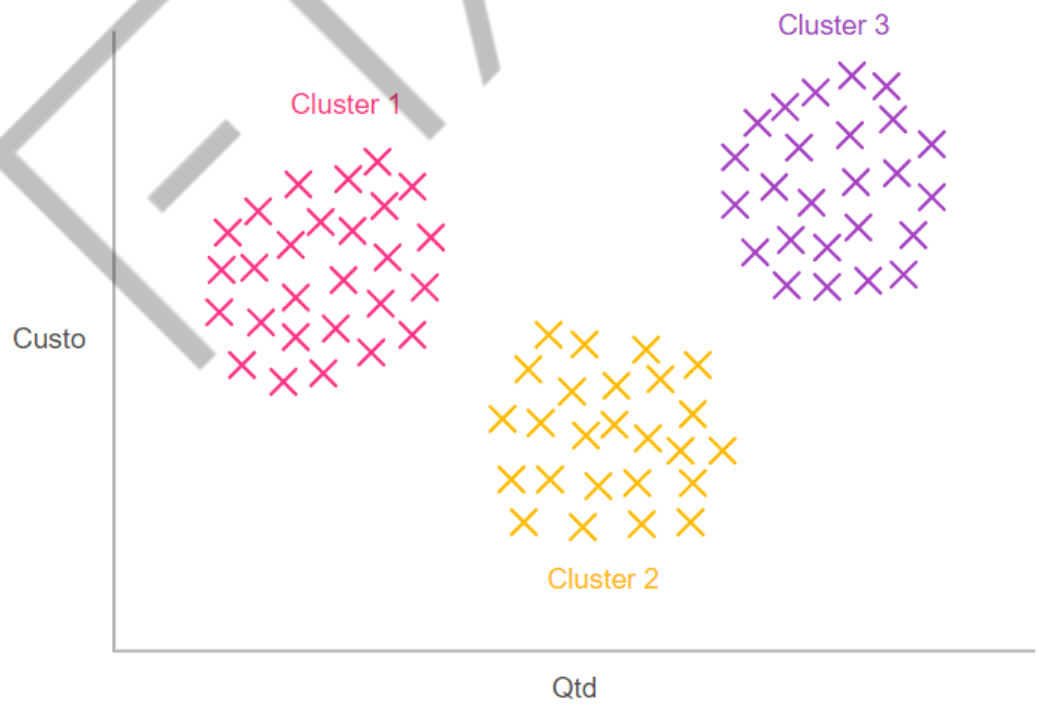

Figura 2 – Exemplo de análise de *cluster*
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Realizar esse tipo de análise é bastante simples. Porém, antes de apresentar as etapas para obtenção dos conglomerados, vale explicarmos o conceito de similaridade e a diferença com outro conceito importante denominado correlação.

1.2.1 Diferenças entre similaridade e correlação

Conforme apresentamos anteriormente, uma análise de *cluster* agrupará indivíduos por meio das respectivas similaridades. Contudo, isso pode gerar algumas dúvidas sobre **similaridade** estar relacionada com **correlação**. Do ponto de vista analítico, temos diferenças significativas entre esses termos.

Em relação à **correlação**, pode ser definida como um coeficiente que mede a força e direção do relacionamento linear entre duas variáveis, conforme apresentam Warrens (2008) e Altman e Krzywinski (2015). Os principais índices que medem a correlação possuem uma escala que pode variar entre -1 e 1.

No caso da **similaridade**, conforme apresenta Hair *et al.* (2009), utilizamos como uma medida de proximidade, ou seja, devemos comparar pares de objetos e aplicar medidas de distâncias, essas métricas verificam o quanto um indivíduo é diferente do outro.

Por exemplo, buscamos medir qual a diferença entre uma pessoa com olhos azuis e outra com olhos castanhos. Nesse caso, a medida de correlação não nos trará as respostas desejadas; dessa forma, utilizamos medidas de distâncias e teremos resultados coerentes, sendo possível informar corretamente o quanto esses indivíduos são diferentes.

Desse modo, conforme levanta Maroco (2010), quando buscamos agrupar variáveis (as colunas do banco de dados) utilizamos a correlação. Quando necessitamos comparar indivíduos (as linhas do banco de dados) devemos aplicar métricas de similaridade, cujas medidas de distância serão primordiais. Vamos agora entender as etapas.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3 Etapas para realizar uma análise de *cluster*

Após o entendimento dos conceitos iniciais sobre análise de *cluster*, vamos entender o processo de realização das análises. As etapas a serem apresentadas foram baseadas nas proposições de Hair *et. al.* (2009) e Maroco (2010); no caso, teremos 5 etapas essenciais, são elas:

1. Escolha das variáveis, identificação de *outliers*, padronização.
2. Medida de distância (similaridade).
3. Escolha do algoritmo de agrupamento (métodos hierárquicos e não hierárquicos).
4. Escolha da quantidade de grupos.
5. Interpretação dos grupos

Dividiremos a explicação de cada etapa em subtópicos.

1.3.1 Escolha das variáveis, identificação de *outliers* e padronização.

Na primeira etapa, temos 3 atividades: selecionar as variáveis, identificar valores discrepantes (*outliers*) e padronização. A seleção das variáveis é uma decisão combinada entre as áreas de negócios e *Analytics*. Identificação de *outliers* é uma etapa importante para encontrar nos dados valores acima ou abaixo do considerado normal. Um ponto interessante é que algumas vezes a própria análise de *cluster* acabada encontrando grupos cujos valores fora do normal fazem parte deles.

A tarefa mais importante dessa primeira etapa é a padronização das variáveis. Em diversos momentos, as colunas possuem unidades de medidas diferentes. Por exemplo, numa análise de produtos, em uma coluna existem as quantidades que o produto foi comprado e outro o valor total gasto. Nesse caso, não podemos aplicar o algoritmo diretamente, precisamos ajustar esses dados.

Temos diversas formas de padronizar os dados, porém os métodos mais utilizados são dois: o primeiro, gerar uma escala entre 0 e 1; o segundo, manter os valores com média 0 e desvio padrão 1. Na expressão 1, temos o método *Min-Max*

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

(escala 0 a 1) e na expressão 2 é apresentado o método da normalização (média 0 e desvio padrão 1).

$$Min - Max = \frac{x - \min(x)}{\max(x) - \min(x)}. \quad (1)$$

$$Norm = \frac{x - \bar{x}}{s}. \quad (2)$$

Em que, $\min(x)$ é o valor mínimo de x, $\max(x)$ é valor máximo de x, $\bar{x}$ é a média e s é o desvio padrão. Não há uma regra padrão sobre qual dos métodos deve ser utilizado, o analista pode aplicar ambos, avaliar a solução dos *clusters* e verificar qual trouxe os melhores resultados.

1.3.2 Medida de distância (similaridade).

A segunda etapa é aplicar a medida de distância, a métrica mais utilizada em análise de *cluster* é a distância euclidiana. Essa métrica é uma generalização do teorema de Pitágoras. Na figura “Conceito distância euclidiana”, apresentamos como funciona a medida.

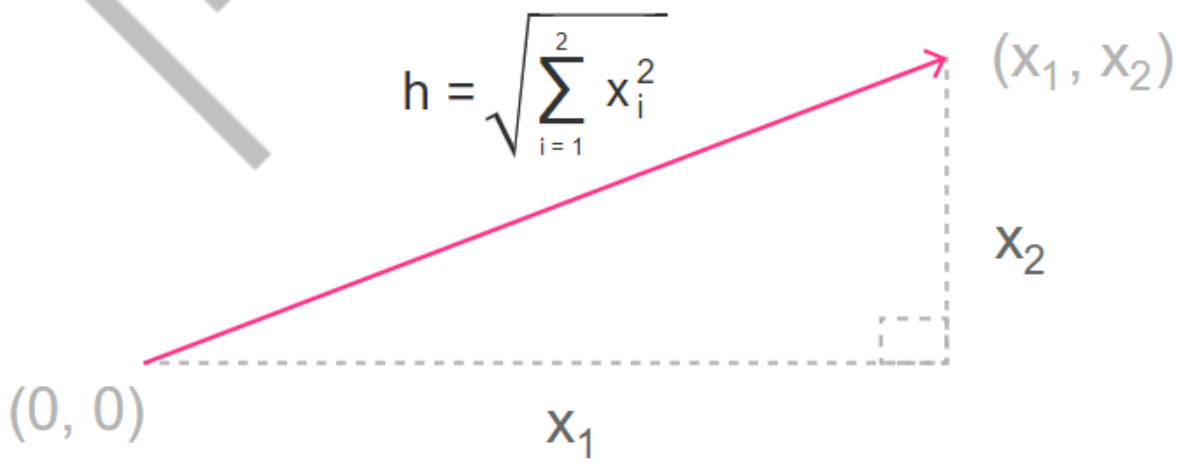

Figura 3 – Conceito distância euclidiana
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Por exemplo, imagine que estamos comparando dois indivíduos quanto à renda, daí utilizamos x_1 para ser a renda do indivíduo 1 e x_2 a renda do indivíduo 2, essa diferença (distância) será representada pela linha vermelha, no caso, a hipotenusa. Na expressão 3, apresentamos a função aplicada nos diversos *softwares* de análise de dados.

$$D_{ij} = \sqrt{\sum_{k=1}^{p} \left(x_{ik} - x_{jk}\right)^2}. \tag{3}$$

1.3.3 Escolha do algoritmo de agrupamento (hierárquicos e não hierárquicos).

Uma vez que podemos obter a diferença entre dois indivíduos a partir das medidas de distância, o próximo passo é buscar uma forma de organizar cada indivíduo no seu respectivo grupo. Dessa forma, na terceira etapa, devemos definir o método de agrupamento.

Temos 2 tipos gerais denominados **hierárquicos** e **não hierárquicos**. Além desses, podemos utilizar um método que combina ambos, o método híbrido. Neste tópico, explicaremos as diferenças entre ambos. Na aplicação prática no *Python,* utilizaremos o método híbrido, assim será possível expor ambos os métodos utilizando as funções de programação.

Conforme Maroco (2010), os algoritmos hierárquicos geram os grupos de forma iterativa. Em cada passo, observam-se as distâncias entre os múltiplos indivíduos e as menores distâncias são agrupadas formando os *clusters*. No final, um coeficiente de aglomeração é obtido informando em cada iteração como os grupos foram ajustados.

Dessa forma, os métodos hierárquicos possuem um papel importante para auxiliar os analistas a definir o número ideal de grupos. Diversos algoritmos foram propostos para resolver esse tipo de problema.

Na figura “Algoritmos de *cluster* hierárquico”, apresentamos os três mais conhecidos. O algoritmo MONA é aplicado em casos de variáveis categóricas binárias,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

o AGNES e o DIANA podem ser utilizados em escalas intervalar ou razão. A diferença entre o AGNES e o DIANA é a forma de inicialização do algoritmo.

No AGNES, cada indivíduo é um grupo formado por ele mesmo, depois ele será combinado em algum outro pela distância. No DIANA, todos os indivíduos fazem parte de um grande grupo e depois são separados, criando os grupos pelas semelhanças entre eles. Não temos uma regra para definir qual o melhor, torna-se importante testar nos dados e verificar qual deles trará os melhores resultados.

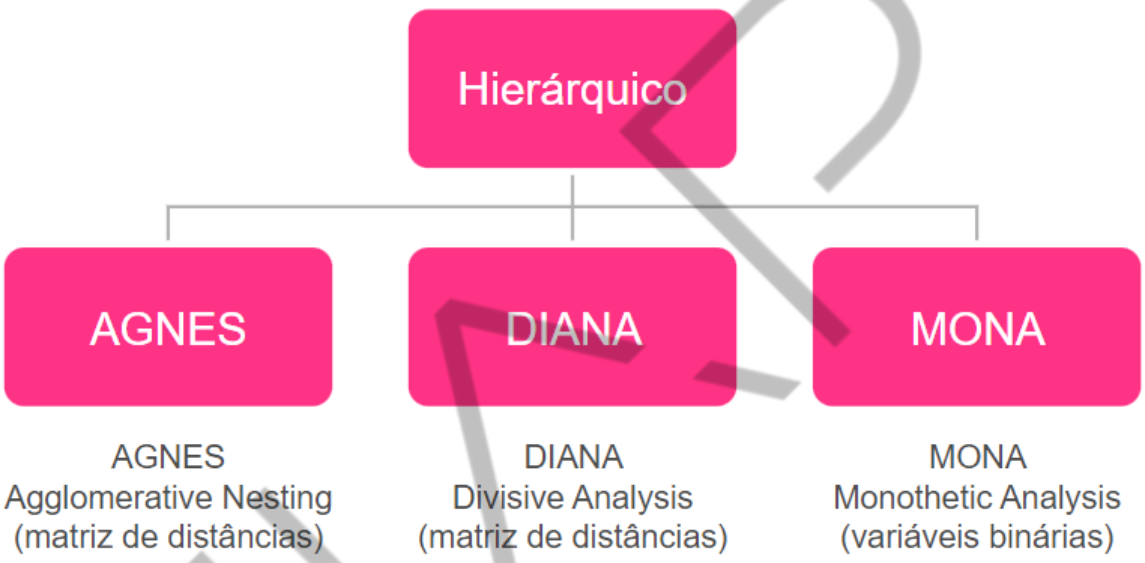

Figura 4 – Algoritmos de *cluster* hierárquico
Fonte: Elaborado pelo autor (2022)

No caso dos não hierárquicos, é fundamental definir inicialmente a quantidade de grupos desejados. Após isso, a cada iteração as distâncias são obtidas e os grupos são formados. A sequência básica de iterações ocorre da seguinte maneira:

1. Seleciona k centroides.
2. Calcula as distâncias para definir os casos próximos aos centroides.
3. Atualiza os centroides.
4. Reassinala todos os casos para os novos centroides.
5. Repete os passos 3 e 4 até convergir.

Uma maneira visual de observar essa sequência pode ser vista no seguinte link: http://shabal.in/visuals/kmeans/1.html. Na figura “Algoritmos de *cluster* não hierárquico”, apresentamos alguns dos principais métodos propostos. Entre eles,

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

destacamos o método CLARA, bastante utilizado em grandes bancos de dados, o algoritmo mais famoso é o *K-means*.

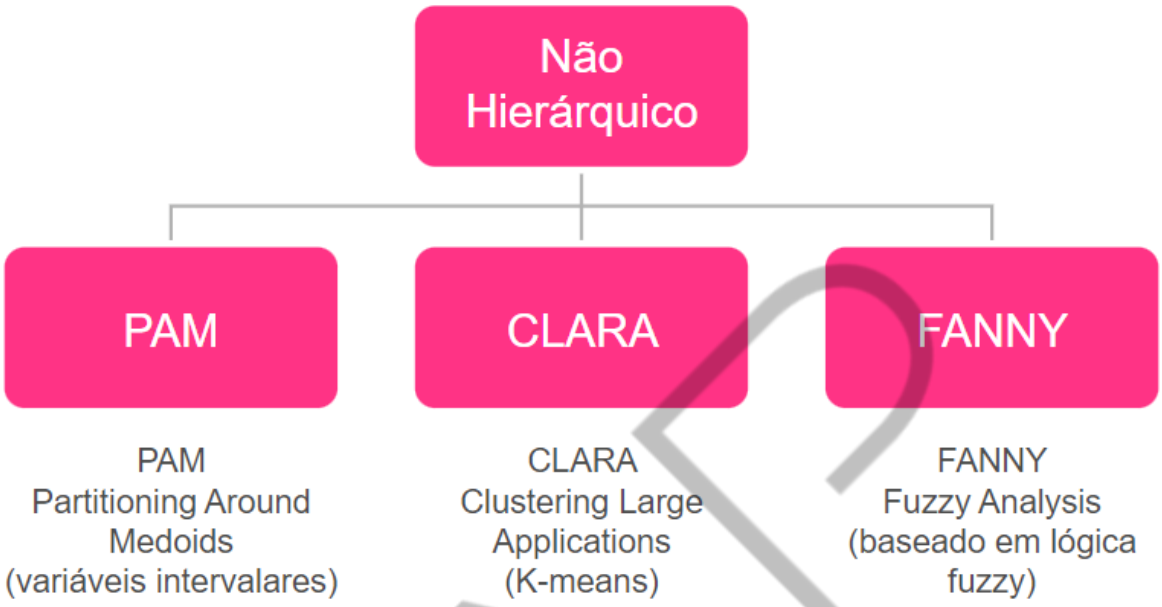

Figura 5 – Algoritmos de *cluster* não hierárquico
Fonte: Elaborado pelo autor (2022)

Cada um dos algoritmos tem seus pontos fortes e fracos. Os métodos hierárquicos auxiliam na seleção do número ideal de grupos, porém o custo operacional para aplicação será alto para realização dos cálculos. Em relação aos não hierárquicos, deve-se presumir já no começo o número de *clusters*. Caso isso não ocorra, o algoritmo não poderá ser aplicado.

Diante disso, a combinação de ambos os métodos é o mais recomendado para uma realização eficiente de uma análise de agrupamentos. A tática mais eficiente é apresentada nos passos a seguir:

1. Utilizar o algoritmo hierárquico para obter o número de grupos.
2. Obter os centroides (médias dos grupos).
3. Utilizar o algoritmo não hierárquico usando o número de grupos obtidos no passo 1 e os centroides do passo 2.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.3.4 Escolha da quantidade de grupos

Um detalhe importante é que tanto nos métodos hierárquicos quanto nos algoritmos não hierárquicos temos técnicas para obter o número ideal de *clusters*. Não há um consenso sobre quais as melhores ou mais eficientes. Contudo, historicamente, adotam-se os hierárquicos para auxiliar no processo de obter esse número ideal.

Apesar disso, acreditamos ser importante conhecer os métodos de seleção do número ideal de grupos em ambos os tipos de algoritmos. Portanto, neste tópico, apresentaremos tanto técnicas para os hierárquicos quanto para os não hierárquicos. Neste capítulo, focaremos nas técnicas AGNES e CLARA, uma vez que são as mais utilizadas pelos analistas nas empresas.

No caso AGNES, podemos apresentar as técnicas do “Dendograma” e do “Coeficiente de Aglomeração”. O dendograma é um gráfico que expõe o processo de como ocorreu a criação do *cluster*. Por meio da análise desse gráfico, conseguimos definir o número de grupos. Na figura “Exemplo dendograma” temos um exemplo do gráfico, em que conseguimos observar a criação de 2 grupos. Os valores são os id’s dos indivíduos contidos nos *clusters*.

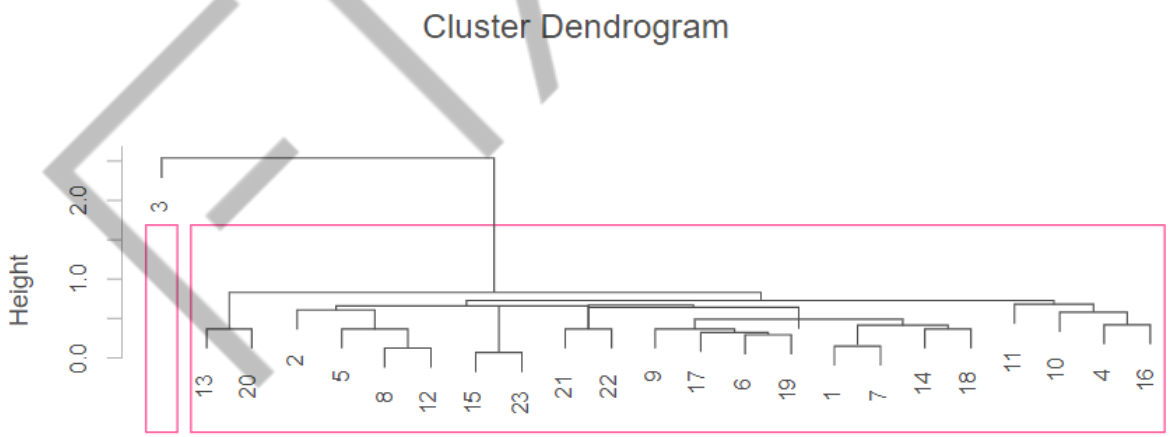

Figura 6 – Exemplo dendograma
Fonte: Elaborado pelo autor (2022)

Outra técnica bastante utilizada é a análise do coeficiente de aglomeração. Essa medida informa o grau de proximidade entre os grupos. Caso a distância entre os grupos seja pequena, deve-se agregá-los; caso contrário, mantê-los separados. Na figura “Exemplo coeficiente de aglomeração”, apresentamos a técnica. O objetivo

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

é encontrar as maiores alturas e verificar a solução de *cluster* que melhor representa os dados.

Nesse exemplo, podemos observar soluções contendo 4, 5, 7 e 8 grupos. Pode parecer um pouco estranho o gráfico recomendar tantos grupos, porém isso é bastante comum. Nesses casos, a equipe de analistas e a equipe de negócios devem discutir sobre qual das soluções pode melhor representar os clientes ou qual seria a melhor solução para criar uma promoção para os grupos.

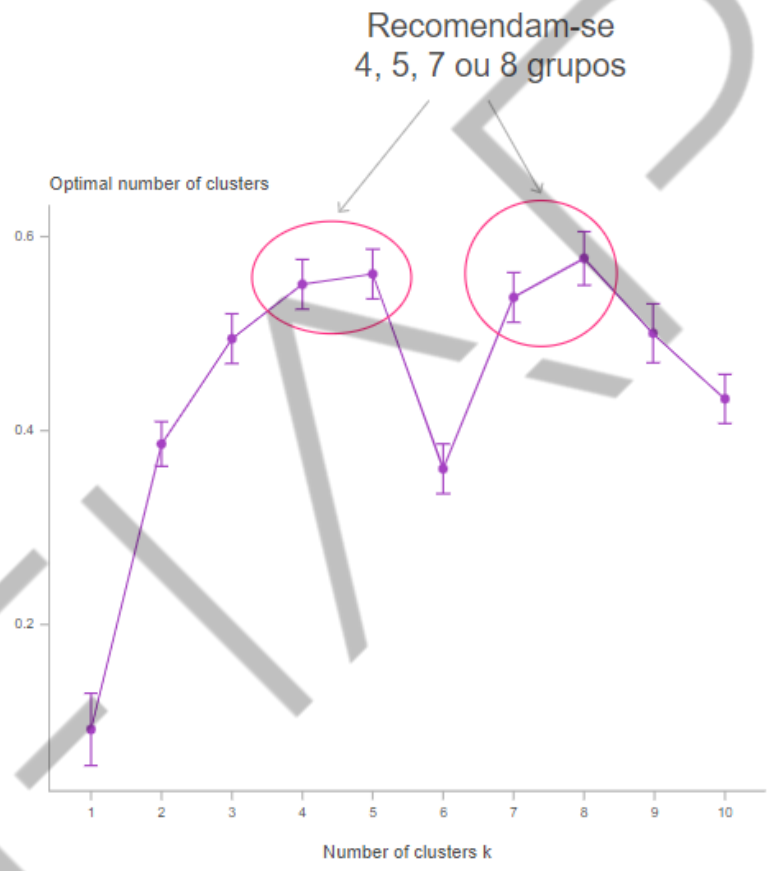

Figura 7 – Exemplo coeficiente de aglomeração
Fonte: Elaborado pelo autor (2022)

Em relação às técnicas relacionadas aos algoritmos não hierárquicos, podemos citar os gráficos WCSS e Silhueta. O gráfico WCSS (Within groups sum of squares) representa a distância entre os grupos. Nesse gráfico, devemos observar quando o decaimento se torna estável. Na figura “Exemplo gráfico WCSS”, temos um exemplo de como podemos selecionar o número de grupos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

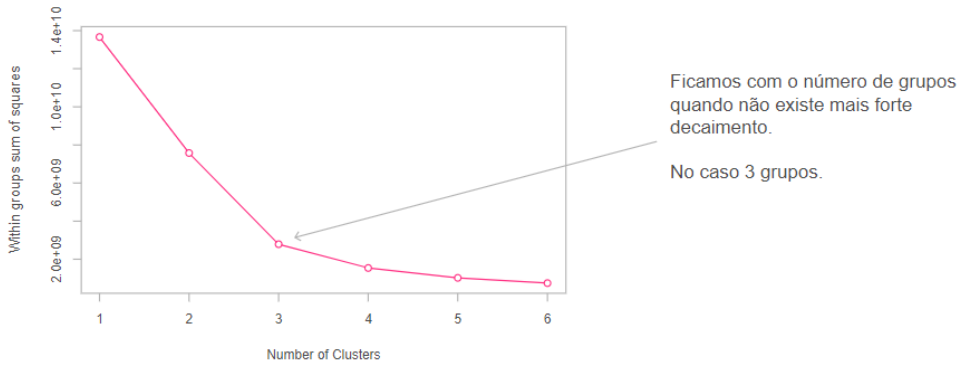

Figura 8 – Exemplo gráfico WCSS
Fonte: Elaborado pelo autor (2022)

No gráfico da silhueta, busca-se observar quais soluções estão mais próximas de 1; uma vez que o coeficiente da silhueta retorna valores na escala entre -1 e 1. Desse modo, quanto mais próximo de 1, mais separados estão os grupos nas soluções. Na figura “Exemplo gráfico da silhueta”, apresentamos um desenho em que 5 grupos foram recomendados para a solução.

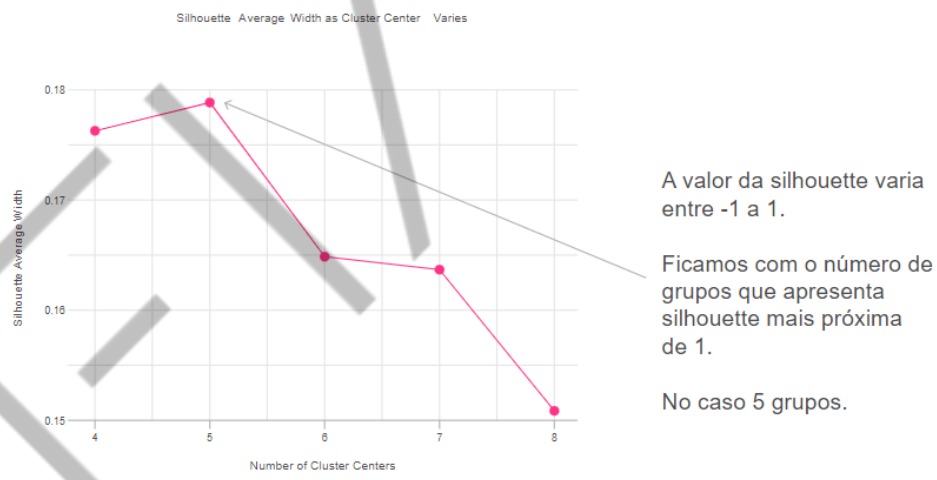

Figura 9 – Exemplo gráfico da silhueta
Fonte: Elaborado pelo autor (2022)

1.3.5 Interpretação dos grupos

Como última etapa, devemos interpretar os grupos propostos pela análise. Uma vez que os algoritmos de aglomeração são não supervisionados, não temos uma medida direta para avaliar se a solução apresentada foi assertiva ou não. Diante disso, novamente uma discussão entre a equipe de negócios e analistas deverá ser organizada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Outro ponto a ser ressaltado para os analistas de dados é a forma de apresentar as soluções escolhidas, uma vez que demonstrar os resultados de uma análise de agrupamentos somente pelos cálculos e modelos pode ser uma tarefa complicada para pessoas não técnicas.

Desse modo, uma forma de traduzir os resultados é a criação de personas que representam as características gerais dos grupos. Esse resultado pode ser atingido, por exemplo, ao combinar os grupos formados pelas variáveis intervalares e combinar com variáveis qualitativas, como gênero, faixa etária e faixa de renda.

Um *case* bastante conhecido no mercado é a solução proposta pela Serasa Experian, o Mosaic (https://www.serasaexperian.com.br/solucoes/mosaic-business/). Nesse produto, existe tanto uma combinação das técnicas de aglomeração quanto a criatividade das equipes de mídia.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 APLICAÇÕES PRÁTICAS EM ANÁLISE DE CLUSTER

Nos próximos tópicos, serão trabalhados em mais detalhes os conceitos citados anteriormente. Além disso, aplicaremos algumas das ferramentas como exemplo prático no *Python*.

Desse modo, vamos conhecer as principais funções, os dados e aplicar a análise de *cluster* nos dados propostos.

2.1 Principais funções no *Python*

Como ferramentas necessárias, os módulos *numpy* e *pandas* serão as principais bibliotecas do *Python* novamente. As bibliotecas *Matplotlib* e *Seaborn* serão novamente utilizadas para análise gráfica.

Para realizar a análise de *cluster,* precisaremos utilizar ferramentas visando padronizar os dados, aplicar os algoritmos hierárquicos e não hierárquicos. Quanto à padronização, temos as funções *scale* e *MinMaxScaler* do módulo *Scikit-learn*. A função "*sch*" do módulo *scipy* será importante para obter o dendograma. As funções AgglomerativeClustering e KMeans do *Scikit-learn* serão utilizadas para aplicar os algoritmos hierárquicos e não hierárquicos, respectivamente.

No quadro "Ferramentas *Python* ACP", apresentamos um resumo das principais funções que serão utilizadas nos exemplos.

Módulos	Funções	Aplicações
Matplotlib	Várias funções serão utilizadas	Análise gráfica
Seaborn	Várias funções serão utilizadas	Análise gráfica
NumPy	Várias funções serão utilizadas	Análise numérica
Pandas	Várias funções serão utilizadas	Leitura e tratamento de dados
Scipy	sch	Aplicar cluster hierárquico
Scikit-Learn	scale	Normalizar os dados média e desvio padrão
	MinMaxScaler	Padronizar dados escala 0-1
	AgglomerativeClustering	Aplicar cluster hierárquico
	KMeans	Aplicar algoritmo não hierárquico (K-Means)

Quadro 1 – Ferramentas *Python* análise de *clusters*
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.2 Dados e exemplos

Novamente utilizaremos o *Colab,* digitando no navegador https://colab.research.google.com/, criando um novo notebook e iniciando as análises. Nos exemplos a seguir, utilizaremos os dados “rfm_analysis.csv”.

Os códigos para importar os módulos e carregar os dados podem ser vistos no código-fonte “Importar módulos e dados”. Juntamente aos códigos apresentados neste texto, também é possível analisar os códigos de forma mais detalhada no arquivo “cap10_exemplos.ipynb” disponibilizado.

```
# gerais
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# cluster
import scipy.cluster.hierarchy as shc
from sklearn.preprocessing import MinMaxScaler, scale
from sklearn.cluster import AgglomerativeClustering
from sklearn.cluster import KMeans

# carregar dados
dados = pd.read_csv('rfm_analysis.csv')
```

Código-fonte 1 – Importar módulos iniciais e dados
Fonte: Elaborado pelo autor (2022)

No código-fonte “Dados dos exemplos”, podemos observar como obter as cinco primeiras linhas do *dataset* que será utilizado.

```
# dados rfm
dados.head()
```

Código-fonte 2 – Dados dos exemplos
Fonte: Elaborado pelo autor (2022)

Na figura “Dados análise RFM”, podemos observar as cinco primeiras linhas do *dataset*. Temos 4 variáveis, o objetivo é criar *clusters* a partir das variáveis *Recency* (Recência), *Frequency* (Frequência) e *Monetary_Value* (Valor).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

	CustomerID	Recency	Frequency	Monetary_Value
0	12346.0	325	1	77183.60
1	12747.0	22	103	4196.01
2	12748.0	4	4595	33719.73
3	12749.0	22	199	4090.88
4	12820.0	44	59	942.34

Figura 10 – Dados análise RFM
Fonte: Elaborado pelo autor (2022)

A análise de Recência, Frequência e Valor monetário é uma técnica provinda da área de Marketing utilizada para descrever o comportamento de compra dos clientes. Os objetivos de mensurar cada variável são:

- **Recência:** indica quão recente um cliente realizou um pagamento (medida em dias).
- **Frequência:** com que frequência o cliente realiza compras (quantidade comprada).
- **Valor monetário:** quanto os clientes gastam nas compras (medida em $).

Uma vez que essas variáveis são medidas, diversos tipos de análises podem ser realizadas. A análise de cluster utilizará essas variáveis para agrupar indivíduos similares quanto essas características. O diferencial dessa ferramenta é a capacidade de encontrar nichos de mercado não observados por técnicas tradicionais de *Marketing*.

Nos próximos tópicos, apresentaremos as análises seguindo as etapas para aplicar o conceito de cluster híbrido, ou seja, combinaremos os algoritmos hierárquicos e não hierárquicos para segmentar essa base de clientes.

2.3 Padronizar os dados

Conforme foi descrito nas etapas para realização de uma análise de *cluster*, vamos iniciar pela padronização das variáveis. O analista deve escolher alguma opção de padronizar as variáveis quando as colunas possuem unidades de medida

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

diferentes. Neste exemplo, apresentaremos a Min-Max e a normalização por média 0 e desvio padrão 1. No código-fonte “Exemplos padronização das variáveis”, apresentamos o processo para realizar a padronização.

```
# selecionar colunas
cols = ['Recency', 'Frequency', 'Monetary_Value']
dados_cluster = dados[cols]

# min-max
minmax_scaler = MinMaxScaler()
dados_minmax = minmax_scaler.fit_transform(dados_cluster)
dados_minmax.shape

# normalizar
dados_normalizados = scale(dados_cluster)
dados_normalizados.shape
```

Código-fonte 3 – Exemplos padronização das variáveis
Fonte: Elaborado pelo autor (2022)

Após o uso dessas funções para padronizar as variáveis, os dados estão num formato de matriz *numpy*. Uma opção interessante é transformá-los em pandas *dataframe*. No código-fonte “*NumPy* para *dataframe* dados padronizados”, demonstramos o processo para adaptar o formato dos dados. Recomenda-se executar os códigos em blocos diferentes do Colab, para visualizar os resultados apresentados nas figuras “Dados padronizados *Min-Max*” e “Dados padronizados *scale*”.

```
# min-max dataframe
dados_minmax = pd.DataFrame(dados_minmax, columns=dados_cluster.column
s)
dados_minmax['CustomerID'] = dados['CustomerID']
dados_minmax.head()

# scale dataframe
dados_normalizados = pd.DataFrame(dados_normalizados, columns=dados_cl
uster.columns)
dados_normalizados['CustomerID'] = dados['CustomerID']
dados_normalizados.head()
```

Código-fonte 4 – *NumPy* para *dataframe* dados padronizados
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Recency	Frequency	Monetary_Value	CustomerID
0.466954	0.000000	0.297242	12346.0
0.031609	0.013000	0.016146	12747.0
0.005747	0.585521	0.129850	12748.0
0.031609	0.025236	0.015741	12749.0
0.063218	0.007392	0.003615	12820.0

Figura 11 – Dados padronizados *Min-Max*
Fonte: Elaborado pelo autor (2022)

Recency	Frequency	Monetary_Value	CustomerID
1.915721	-0.410450	10.066906	12346.0
-0.718516	0.057911	0.311637	12747.0
-0.875005	20.684175	4.257675	12748.0
-0.718516	0.498722	0.297586	12749.0
-0.527251	-0.144127	-0.123237	12820.0

Figura 12 – Dados padronizados *scale*
Fonte: Elaborado pelo autor (2022)

Uma vez que os dados foram padronizados, escolheremos um tipo específico e continuaremos as análises. No caso, vamos utilizar os dados normalizados pela média 0 e desvio padrão 1. Para este exemplo, não há diferença de performance entre esses tipos de padronização, somente adotaremos por questões didáticas. Durante um projeto prático, o analista precisa aplicar ambos os métodos e verificar qual deles apresenta melhores resultados.

2.4 Obter número de *clusters*: algoritmos hierárquicos

O próximo passo após padronizar as variáveis é definir o número de grupos. Neste exemplo encontraremos o número de *cluster* de forma direta; contudo, nas

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

aplicações diárias, o analista deverá realizar várias tentativas até encontrar um valor minimamente aceitável e testar diversas soluções.

Primeiramente, vamos aplicar o método do dendograma. Um detalhe a ser relembrado é que os métodos hierárquicos não devem ser realizados em dados contendo muitas linhas. Neste caso, iremos primeiro buscar uma amostra de dados, depois executar o algoritmo. No código-fonte “Número de *clusters* - Dendograma”, apresentamos o processo para obter o dendograma; na figura “Dendograma dos dados analisados”, temos os resultados.

```
# algoritmo hierarquico
dados_cluster_hierarquico = dados_normalizados[cols].sample(300, rando
m_state = 5461)
cluster_hier = shc.linkage(dados_cluster_hierarquico, method='ward')

# grafico
plt.figure(figsize=(10, 7))
plt.title("Dendograma")
dend = shc.dendrogram(cluster_hier)
plt.axhline(y=10, color='r', linestyle='--')
```

Código-fonte 5 – Número de *clusters* – Dendograma
Fonte: Elaborado pelo autor (2022)

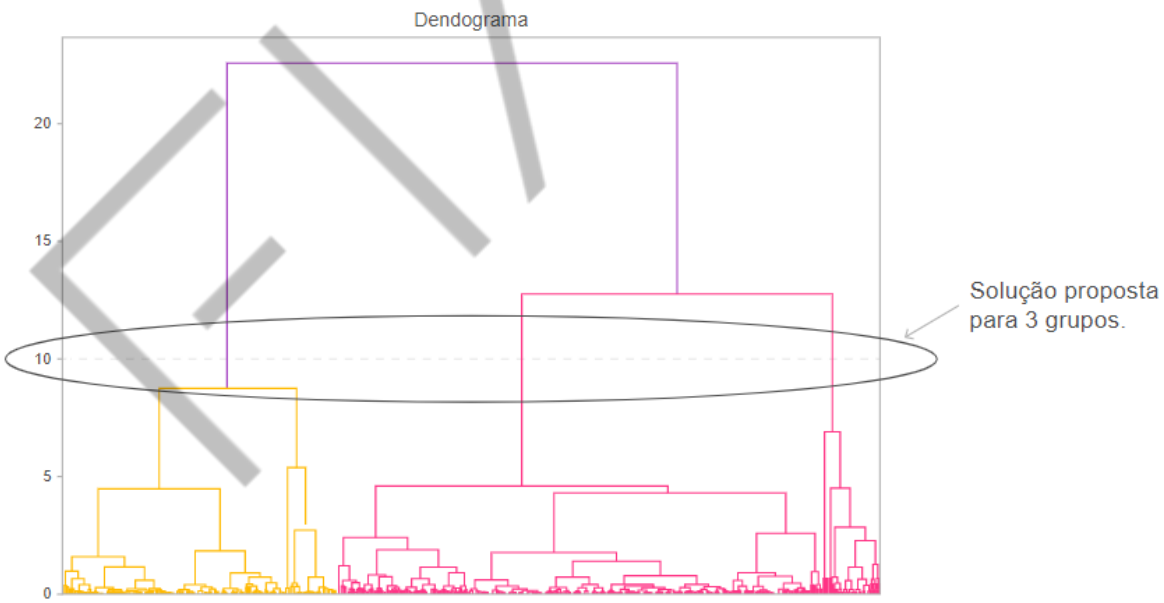

Figura 13 – Dendograma dos dados analisados
Fonte: Elaborado pelo autor (2022)

Nesse exemplo, utilizamos a função *shc.linkage* para realizar o *cluster* hierárquico. A limitação dessa função é a impossibilidade de fornecer de forma facilitada as distâncias do processo de aglomeração. Desse modo, utilizaremos a

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

função *AgglomerativeClustering* para obter as distâncias em cada etapa do processo, depois analisaremos quantos grupos poderão ser definidos.

No código-fonte “Obtenção dos coeficientes de aglomeração”, apresentamos o processo para realizar o cluster hierárquico com a nova função. Na figura “Gráfico coeficientes de aglomeração”, temos os resultados.

```
# algoritmo hierarquico
cluster = AgglomerativeClustering(n_clusters=3, affinity='euclidean',
linkage='ward', compute_distances=True)
cluster.fit(dados_cluster_hierarquico)
distancias_grupos = pd.DataFrame(cluster.distances_, columns = ['dista
ncia'])
distancias_grupos['iteracao'] = distancias_grupos.index
distancias_grupos['qtd_grupos'] = (max(distancias_grupos['iteracao'])
- distancias_grupos['iteracao'])+1

# grafico
n_cluster_test = 15
avaliacao_clusters = distancias_grupos[distancias_grupos['qtd_grupos']
 <= n_cluster_test][['distancia', 'qtd_grupos']]
plt.plot(avaliacao_clusters['qtd_grupos'], avaliacao_clusters['distanc
ia'], '-o')
plt.show()
```

Código-fonte 6 – Obtenção dos coeficientes de aglomeração
Fonte: Elaborado pelo autor (2022)

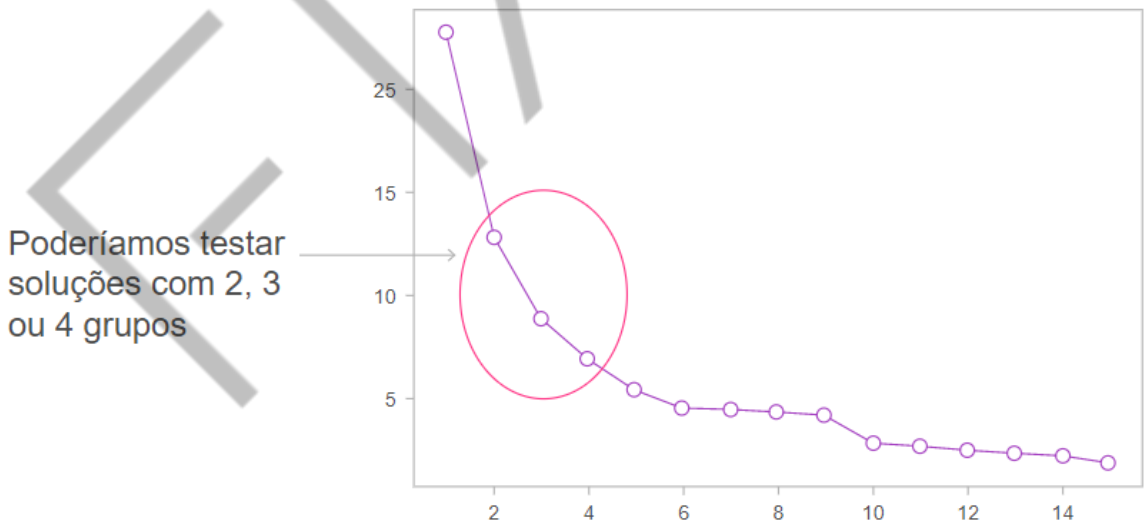

Figura 14 – Gráfico coeficientes de aglomeração
Fonte: Elaborado pelo autor (2022)

Por meio da análise do gráfico, devemos testar soluções contendo 2, 3 ou 4 grupos. Em nosso exemplo, optamos por analisar a solução utilizando 3 grupos. Uma vez que definimos o número de *clusters*, temos que obter os centroides. No código-

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

fonte “Obtenção dos centroides”, apresentamos a forma de obter essas informações. Na figura “Valor dos centroides *cluster* hierárquico”, apresentamos os resultados.

```
# obter grupos
dados_cluster_hierarquico['cluster'] = cluster.fit_predict(dados_clust
er_hierarquico)

# centroides
centroides = dados_cluster_hierarquico.groupby('cluster') \
                     .agg(media_recencia = pd.NamedAgg('Recency', 'mea
n'),
                          media_frequencia = pd.NamedAgg('Frequency',
'mean'),
                          media_valor = pd.NamedAgg('Monetary_Value',
'mean'))
centroides
```

Código-fonte 7 – Obtenção dos centroides
Fonte: Elaborado pelo autor (2022)

cluster	media_recencia	media_frequencia	media_valor
0	1.327466	-0.278879	-0.197289
1	-0.722241	1.872975	0.607411
2	-0.557484	-0.088355	-0.070899

Figura 15 – Valor dos centroides *cluster* hierárquico
Fonte: Elaborado pelo autor (2022)

Após obter os centroides, conseguiremos utilizar esses dados para aplicar o algoritmo não hierárquico.

2.5 Obter os *clusters*: algoritmos não hierárquicos

O próximo passo é obter a solução final de agrupamentos. Essa atividade pode ser realizada pelos algoritmos não hierárquicos. Um ponto a ressaltar seria que, durante a aplicação dos hierárquicos, tivemos que utilizar uma amostra dos dados. No caso dos não hierárquicos, devemos utilizar todos os dados existentes uma vez que o *K-Means* permite esse processo.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Aplicar o algoritmo é bastante simples. No código-fonte “Aplicação do K-Means”, temos o processo. Nesse algoritmo, precisamos definir desde o início a quantidade de grupos, também utilizamos os centroides como inicialização do modelo.

Vale ressaltar que o uso dos centroides é realizado porque estamos seguindo a metodologia do cluster híbrido. Caso estivéssemos aplicando somente o K-Means, não precisaríamos utilizar os parâmetros “init = centroides.values” e “n_init=1”, como podemos perceber no código-fonte “Aplicação do *K-Means*”.

```
# treinamento
cluster_kmeans = KMeans(n_clusters = 3, init = centroides.values, n_in
it=1, max_iter = 1000)
cluster_kmeans.fit(dados_normalizados[cols])

# associar os clusters aos dados
dados['cluster'] = cluster_kmeans.predict(dados_normalizados[cols])
dados.head()
```

Código-fonte 8 – Aplicação do *K-Means*
Fonte: Elaborado pelo autor (2022)

Uma vez que temos o algoritmo não hierárquico treinado, podemos associar aos dados existentes rapidamente pela função *predict*. Um ponto interessante é que, caso novos clientes sejam adicionados na base, podemos classificá-los diretamente pelo uso da função. Na figura “*Clusters* associados aos dados originais”, temos uma amostra de como podemos observar grupos relacionados com os id’s dos clientes.

CustomerID	Recency	Frequency	Monetary_Value	cluster
12346.0	325	1	77183.60	1
12747.0	22	103	4196.01	2
12748.0	4	4595	33719.73	1
12749.0	22	199	4090.88	2
12820.0	44	59	942.34	2

Figura 16 – *Clusters* associados aos dados originais
Fonte: Elaborado pelo autor (2022)

2.6 Interpretação dos *clusters*

Finalmente, como última etapa, devemos analisar os grupos encontrados pelos algoritmos. Conforme foi apresentado no início do capítulo, os modelos de agrupamentos são considerados algoritmos não supervisionados. Dessa forma, após a obtenção dos grupos, o computador não saberá (com a tecnologia atual) interpretar

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

sozinho o que significa cada *cluster*. Diante disso, torna-se necessária a intervenção humana.

Durante essa etapa, recomenda-se maior interação entre os analistas de dados e área de negócios para uma melhor interpretação dos resultados. As análises a serem apresentadas são ferramentas que a área de *Analytics* poderá utilizar nesses momentos.

A primeira análise a ser realizada é contar quantos indivíduos existem em cada grupo. Não é uma regra geral, mas boas soluções de *clusters* tendem a possuir quantidades equilibradas. Caso isso não ocorra, podemos considerar que encontramos nichos de clientes (algo positivo) ou temos *outliers* na base que podem alterar o comportamento do algoritmo (algo negativo).

No código-fonte "Obtenção das quantidades de grupos", apresentamos o processo para contar o número de clientes em cada *cluster*. Na figura "Número de clientes nos *clusters*", temos os resultados.

```
# contagem
dados.groupby('cluster').size().to_frame('n')
```

Código-fonte 9 – Obtenção das quantidades de grupos
Fonte: Elaborado pelo autor (2022)

cluster	n
0	1052
1	15
2	2853

Figura 17 – Número de clientes nos *clusters*
Fonte: Elaborado pelo autor (2022)

Podemos perceber que o grupo 1 possui somente 15 clientes, o que pode indicar *outliers* ou nicho. Precisaremos realizar mais análises para entendermos melhor a configuração desses grupos. No código-fonte "Aplicação de análise qualitativa dos grupos", temos um processo que verifica o quanto a média dos grupos está acima ou abaixo da média geral dos clientes.

```
# centroides dos grupos
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

```
centroides_final = dados.groupby('cluster') \
     .agg(media_recencia = pd.NamedAgg('Recency', 'mean'),
          media_frequencia = pd.NamedAgg('Frequency', 'mean'),
          media_valor = pd.NamedAgg('Monetary_Value', 'mean'))

# medias gerais
media_recencia = np.mean(centroides_final['media_recencia'])
media_frequencia = np.mean(centroides_final['media_frequencia'])
media_valor = np.mean(centroides_final['media_valor'])

# regras de avaliacao
centroides_ajustado = centroides_final.copy()
centroides_ajustado['media_recencia'] = np.where(centroides_final['med
ia_recencia'] > media_recencia, 'alto', 'baixo')
centroides_ajustado['media_frequencia'] = np.where(centroides_final['m
edia_frequencia'] > media_frequencia, 'alto', 'baixo')
centroides_ajustado['media_valor'] = np.where(centroides_final['media_
valor'] > media_valor, 'alto', 'baixo')

# ver os resultados
centroides_ajustado
```

Código-fonte 10 – Aplicação de análise qualitativa dos grupos
Fonte: Elaborado pelo autor (2022)

Na figura “Grupos classificados”, podemos verificar o comportamento dos grupos. O grupo “0” possui alta recência (comprou há muito tempo), baixa frequência (comprou poucas vezes) e baixo valor (gastou pouco). Aparentemente, esse grupo precisa ser mais bem trabalhado pela área de Marketing para torná-los mais atrativos para a empresa.

O grupo “1” tem baixa recência, alta frequência e alto valor. Esse grupo oferece bastante resultado para a empresa, porém possui somente 15 clientes. Nesse caso, uma pesquisa qualitativa com esses indivíduos seria mais interessante para entender o porquê do alto consumo.

O grupo “2” tem baixa recência, baixa frequência e baixo valor. Esses indivíduos podem ser clientes que transitaram na empresa, porém realizaram compras simples de baixo valor. Nesse caso, a área de Marketing poderia trazer soluções para incentivar essas pessoas a voltarem novamente para a empresa.

cluster	media_recencia	media_frequencia	media_valor
0	alto	baixo	baixo
1	baixo	alto	alto
2	baixo	baixo	baixo

Figura 18 – Grupos classificados
Fonte: Elaborado pelo autor (2022)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A análise de cluster é uma técnica bastante útil e complexa a ser aplicada no ambiente de negócios. Os exemplos apresentados neste capítulo não extinguem as estratégias que poderão ser aplicadas para atingir os melhores resultados. Os analistas de dados deverão explorar as diversas alternativas que poderão ser aplicadas.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

ATIVIDADE TREINO – ANÁLISE DE *CLUSTER*

Essa atividade não é avaliativa, caso queira ter um feedback, entre em contato com seu tutor pelo Teams

O arquivo “scouts.csv” foi disponibilizado e utilizaremos as ferramentas para aplicar uma análise de cluster.

O objetivo da análise é avaliar, encontrar jogadores de futebol com estatísticas similares e auxiliar dirigentes a melhor contratar os atletas. As colunas presentes estão dispostas no quadro a seguir:

Código	Significado
FS	Falta Sofrida
PE	Passes Errados
A	Assistências
FT	Finalização na trave
FD	Finalização defendida
FF	Finalização para fora
G	Gol
I	Impedimento
PP	Pênalti Perdido
RB	Roubada de bola
FC	Falta Cometida
GC	Gol Contra
CA	Cartão amarelo
CV	Cartão vermelho
SG	Sem gol sofrido
DD	Defesa Difícil
DP	Defesa de pênalti
GS	Gol sofrido

Para esta atividade, analise os dados propostos e responda às perguntas. Ao final, deve ser entregue um arquivo “.ipynb”, contendo os códigos e as respostas.

1. Acesse o *Colab*, carregar os módulos e os dados necessários.

2. Realize uma preparação de dados:

a) Filtre as colunas necessárias. As colunas são: FS, PE, A, FT, FD, FF, G, I, RB, FC, CA, CV.

b) Normalize os dados (Min-Max).

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3. Defina o número de clusters. Utilizar o método do dendograma.

4. Associe os clusters aos dados originais.

5. Conte quantos indivíduos estão contidos nos grupos.

6. Quais outras análises você pode propor?

Importante: as respostas estão disponibilizadas no arquivo anexo ".ipynb".

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

ALTMAN, N.; KRZYWINSKI, M. Association, correlation and causation. **Nature Methods**. Vol. 12, n. 10, 2015. p. 899-900. Disponível em: https://www.nature.com/articles/nmeth.3587. Acesso em: 03 jan. 2022.

HAIR, J. F.; BLACK, W. C.; BABIN, B. J.; ANDERSON, R. E.; TATHAM, R. L. **Análise multivariada de dados**. São Paulo, Bookman editora, 2009.

HENNIG, C.; MEILA, M.; MURTAGH, F.; ROCCI, R. **Handbook of cluster analysis**. Nova Iorque: CRC Press, 2015.

KAUFMAN, L.; ROUSSEEUW, P. J. **Finding groups in data: an introduction to cluster analysis**. New Jersey: John Wiley & Sons, 2009.

MAROCO, J. **Análise estatística com utilização do SPSS.** Lisboa: Sílabo, 2010.

WARRENS, M. J. On association coefficients for 2×2 tables and properties that do not depend on the marginal distributions. **Psychometrika**. Vol. 73, n. 4, 2008. p. 777-789. Disponível em: https://link.springer.com/content/pdf/10.1007/s11336-008-9070-3.pdf. Acesso em: 03 jan. 2022.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com